TRIANZ℠ | ACCELERATING DIGITAL EVOLUTION

Application Architecture

Great Place To Work. Certified 2023 USA | INDIA

Purpose of this document

Priority team along with Trianz and AWS is working on a digital transformation project for their day-to-day operations which are currently supported by the following platforms. The scope of this document is to cover the Fleet management solution.

AMCS platform has capabilities (like CRM, Financials) also that will be migrated to Netsuite, which is not covered in this document.

This document covers the architecture to support the solution that would be built on AWS, which includes the solution for

- Customer Access (Web and Mobile)
- Driver App (Android App)
- Portal for Fleet operations (scheduling and dispatching)
- Platform management
- AWS backend to support these applications

Tech Stack

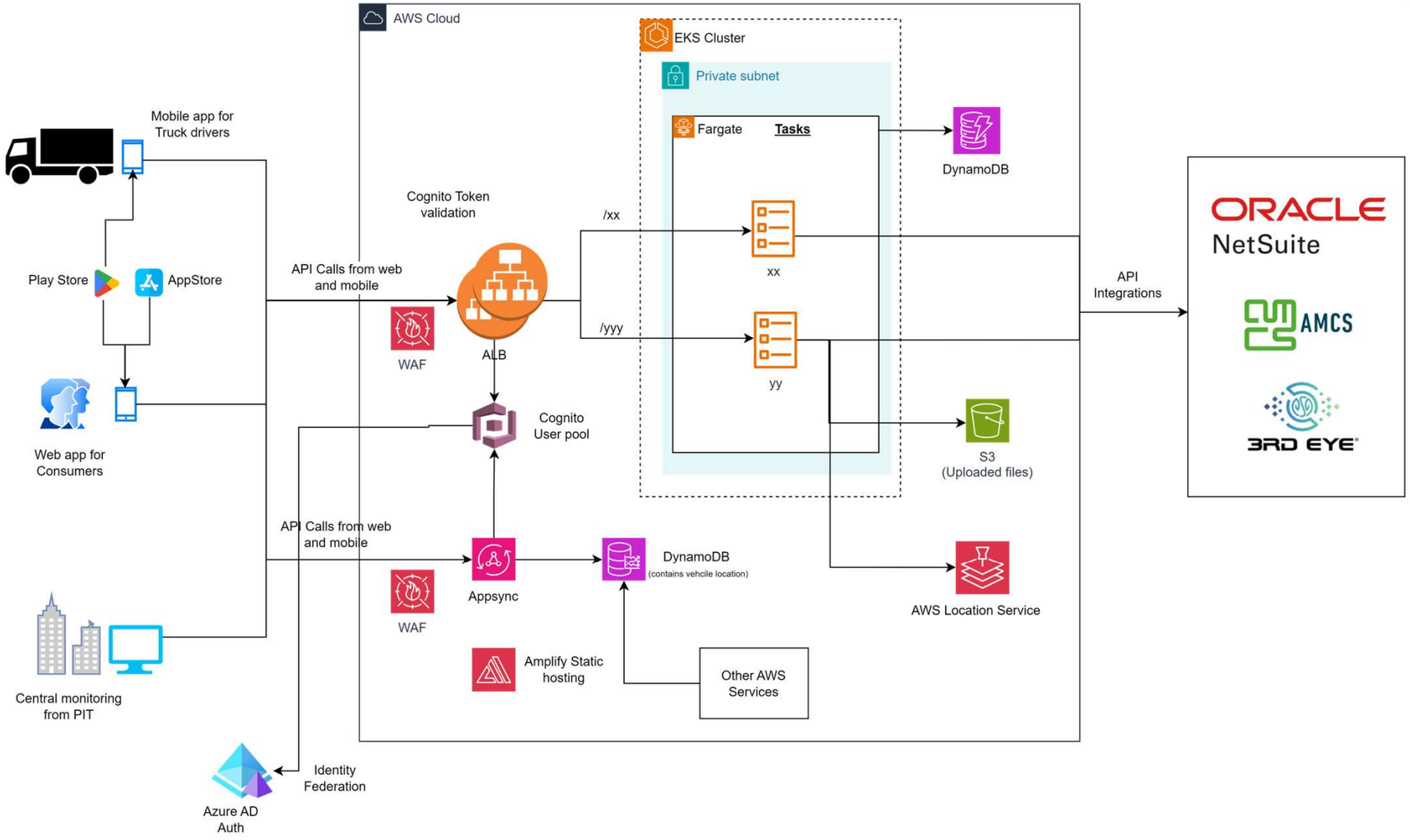
Stack Name	Service Description	Vendor Name	Purpose
AWS Amplify	A set of tools and services that help you build web and mobile apps with AWS. It allows you to deploy frontend apps with any web framework, use TypeScript for fullstack development, and scale to millions of users with zero cloud expertise.	Amazon Web Services (AWS)	Hosting the static content of web applications
EKS	A managed service that eliminates the need to install, operate, and maintain your own Kubernetes control plane on AWS. It automates the deployment, scaling, and maintenance of containerized applications.	Amazon Web Services (AWS)	Backend for APIs hosted in containers managed by EKS.
Appsync and GraphQL subscription operations	Allows you to utilize subscriptions to implement live application updates and push notifications. When clients invoke the GraphQL subscription operations, a secure WebSocket connection is automatically established and maintained.	Amazon Web Services (AWS)	Provides real-time updates to the frontend based on events from different systems.
Azure AD auth	A cloud-based service that acts as the trust broker between your on-premises Exchange organization and the Exchange Online organization. It provides secure and scalable validation of passwords against your on-premises Active Directory.	Microsoft	Current Authentication platform for Priority. It serves as a central identity system for SSO (Single Sign-On) solutions.
Global accelerator	A networking service that helps you improve the availability, performance, and security of your public applications. It provides two global static public IPs that act as a fixed entry point to your application endpoints.	Amazon Web Services (AWS)	It provides a single entry point for the EKS cluster for different regions.
Route 53	A highly available and scalable Domain Name System (DNS) web service. It connects user requests to internet applications running on AWS or on-premises.	Amazon Web Services (AWS)	Resolves the DNS for all the services hosted on the internet.
AWS OpenSearch Service	A managed service that makes it easy to deploy, operate, and scale OpenSearch clusters in the Amazon Cloud. It supports OpenSearch and legacy Elasticsearch OSS (up to version 7.10).	Amazon Web Services (AWS)	Observability of the EKS cluster.
Karpenter auto-scaler	An open-source, flexible, high-performance Kubernetes cluster autoscaler built with AWS. It helps improve your application availability and cluster efficiency by rapidly launching right-sized compute resources in response to changing application load.	Amazon Web Services (AWS)	Auto-scaling of the Kubernetes nodes.

Tech Stack

Stack Name	Service Description	Vendor Name	Purpose
App Mesh	A service mesh that provides application-level networking to make it easy for your services to communicate with each other across multiple types of compute infrastructure. It standardizes how your services communicate, giving you end-to-end visibility and ensuring high availability for your applications.	Amazon Web Services (AWS)	Provides end-to-end visibility and ensures high availability for your applications.
Docker	A platform that helps developers build, share, and run applications using containers. It enables you to separate your applications from your infrastructure so you can deliver software quickly.	Docker, Inc.	Enables to deliver software quickly by isolating applications in containers.
ALB (Application Load Balancer)	A service that automatically distributes incoming application traffic across multiple targets, such as Amazon EC2 instances, containers, and IP addresses, in one or more Availability Zones.	Amazon Web Services (AWS)	Distributes incoming application traffic across multiple microservices in EKS cluster to ensure high availability.
WAF (Web Application Firewall)	A security tool that helps protect web applications by filtering and monitoring HTTP traffic between a web application and the Internet. It typically protects web applications from attacks such as cross-site forgery, cross-site-scripting (XSS), file inclusion, and SQL injection.	Various (including AWS, Cloudflare)	Protects public ALB from common web exploits and attacks, ensuring security.
DynamoDB	A serverless, NoSQL, fully managed database with single-digit millisecond performance at any scale. It addresses your needs to overcome scaling and operational complexities of relational databases.	Amazon Web Services (AWS)	Offers scalable, high-performance NoSQL database for backend data stored in AWS
S3	An object storage service offering industry-leading scalability, data availability, security, and performance. It allows you to store and retrieve any amount of data at any time, from anywhere.	Amazon Web Services (AWS)	Storage of the images, files and content needed for the platform
Cognito	An identity platform for web and mobile apps. It's a user directory, an authentication server, and an authorization service for OAuth 2.0 access tokens and Amazon credentials.	Amazon Web Services (AWS)	Authentication of the internal users and customers.

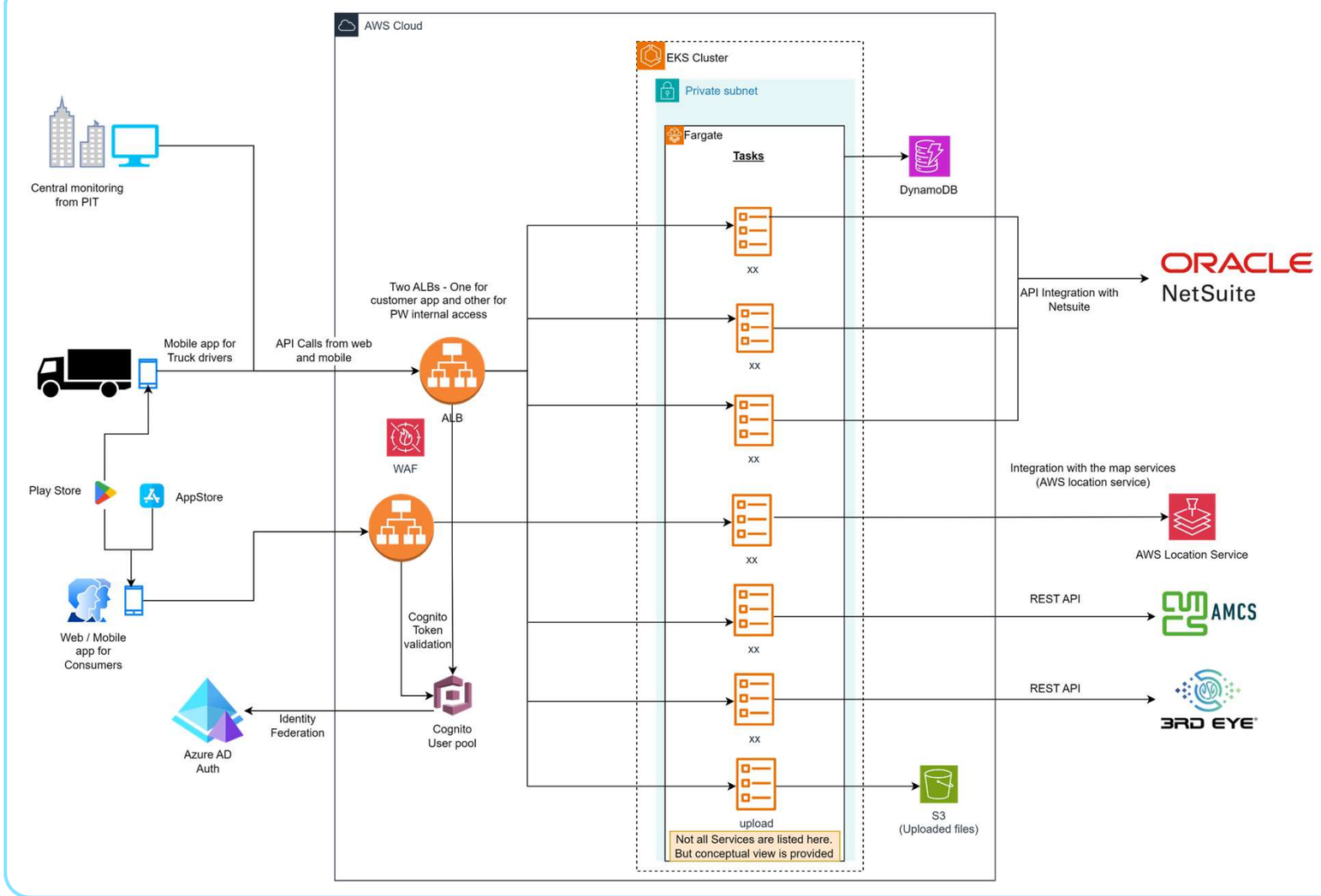
High Level Application Architecture

- The application will have three different personas accessing.
  - Customers (web and mobile app)
  - Employees from the PIT (web)
  - Driver (Android app)
- Application components will be segregated for the access from customers and from internal employees (including drivers)
- AWS Amplify will be used for static content hosting
- Apis will be hosted on EKS / Fargate. Microservices on EKS will integrate with the other AWS services and the Saas platform to retrieve the data
- To address the requirement of real-time / offline data synchronisation, design includes Appsync and GraphQL subscription operations



Microservices and APIs

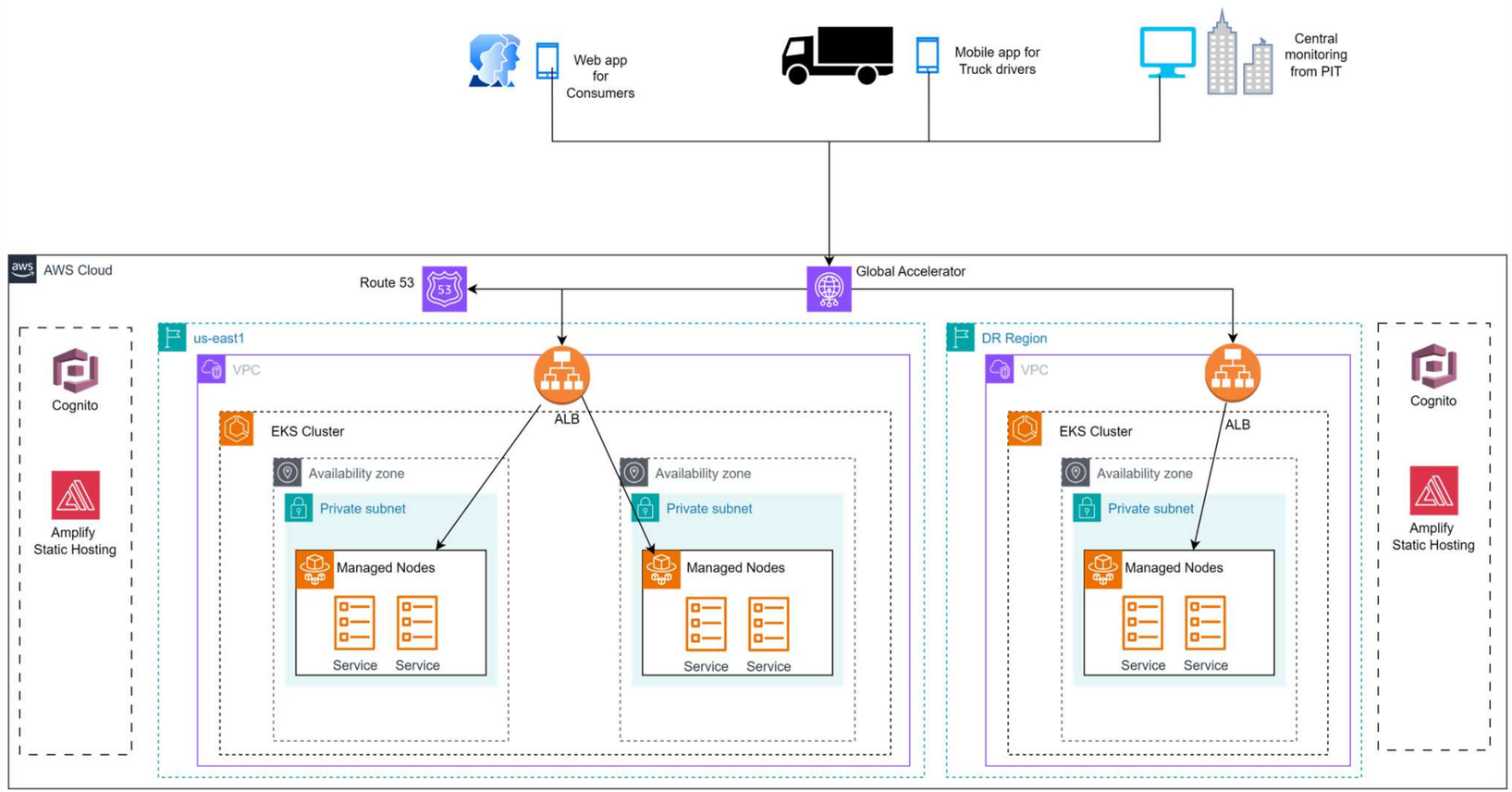
- A common EKS cluster will be used to service the APIs request for customers and internal Priority users
- In order to segregate the traffic from the external users and the internal users, two ALBs will be deployed one for the customer APIs and other for the internal Priority APIs.
- All customer records must be created in Cognito, upon validation of customer record from the Netsuite platform (Dependency: customer's phone number or email should be available in the Netsuite platform)
- All the employees (including drivers) will be authenticated using the Azure AD credential (SSO). Separate Cognito user pool should be created for employees, which will allow only Azure AD authentication.
- Services are segregated for customers and internal users as the functionality of these services will be quite distinct



DR for Application

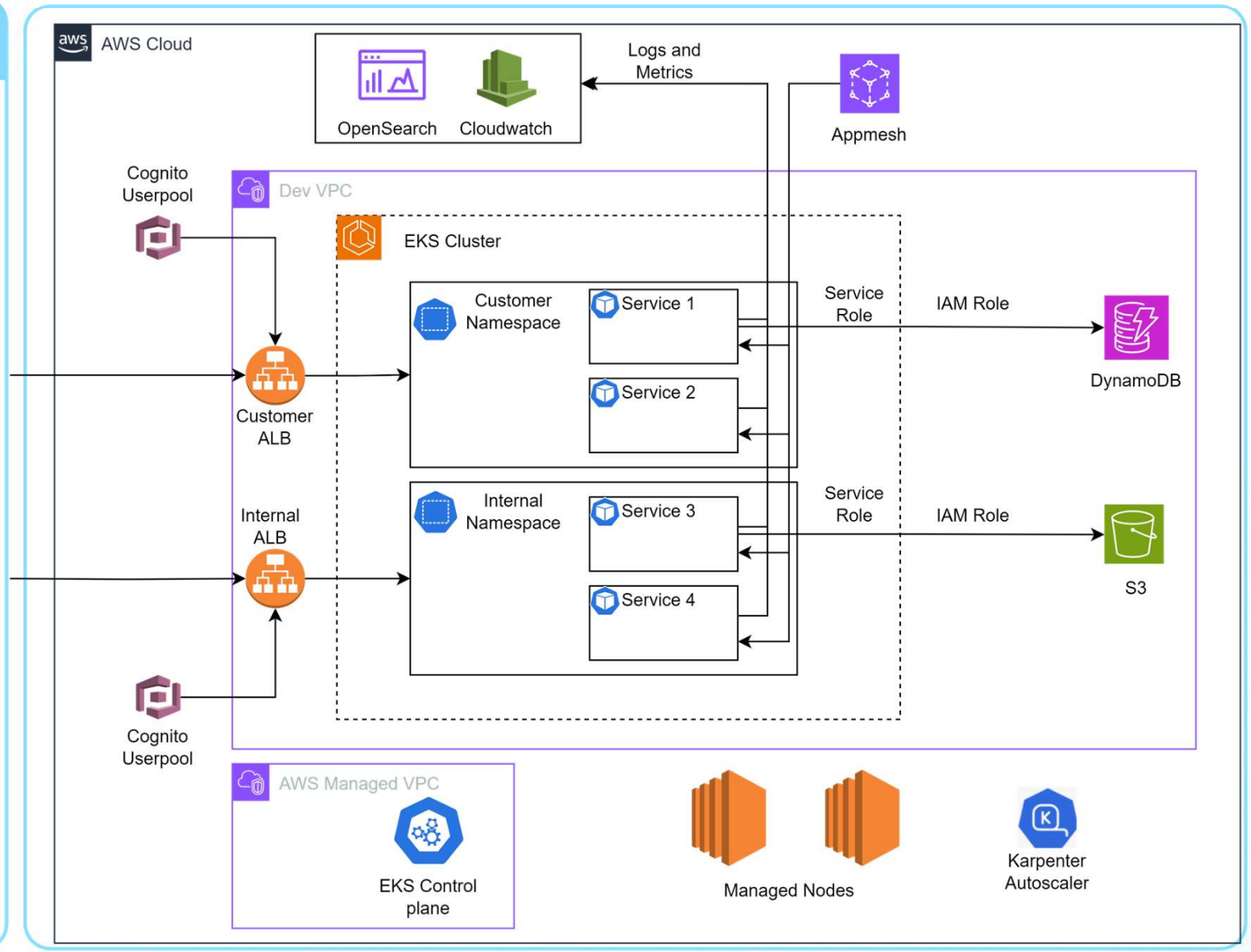
In order to provide disaster recovery in case of primary region failure, multi-region solution must be built using following components

- Deploy EKS clusters in multiple regions
- Global accelerator provides statis IP Addresses that route traffic to the nearest healthy endpoint. Create end point groups and health checks for the EKS cluster
- Deploy front end application using AWS amplify in multiple regions.
- Create Cognito user pools in multiple regions to handle user authentication. Ensure user data is replicated across the regions.
- Use DynamoDB global tables to replicate data across regions.
- While Global accelerator provides static Ip addresses and directs the traffic to the nearest healthy endpoint, Route 53 complements this by managing DNS records.



Kubernetes Internals

- This slide illustrates the internal structure of each EKS cluster, with only one shown for clarity.
- Each cluster will have two namespaces: customer and internal.
- For customers:
  - A dedicated ALB, Cognito user pool, and namespace will be deployed to service the APIs.
- For internal purposes:
  - A separate ALB, Cognito (with SAML integration with Azure AD), and internal namespace will be deployed.
- Service roles for each service will be mapped to IAM roles to provide access to AWS services like DynamoDB and S3 bucket.
- All logging from the microservices will be routed to CloudWatch.
- AWS OpenSearch Service will be leveraged for observability.
- The services of the EKS cluster will run on managed nodes.
- Karpenter auto-scaler must be leveraged for dynamic provisioning of new nodes of need basis.
- App Mesh provides application-level networking to make it easy for your services to communicate with each other
- EKS cluster itself will run on a fully managed VPC to enable communication with the on-prem or internet



DNS Management

Domain Hosting and Management:

- The primary record for the domain prioritywaste.com is hosted on Cloudflare (this is already in place).

Subdomain Configuration:

- The following subdomains will be directed to corresponding environments on AWS:
  - **dev.prioritywaste.com** for the development environment.
  - **uat.prioritywaste.com** for the user acceptance testing environment.
  - **prod.prioritywaste.com** for the production environment.
  - app.prioritywaste.com reserved for future use.

Environment-Specific Subdomains:

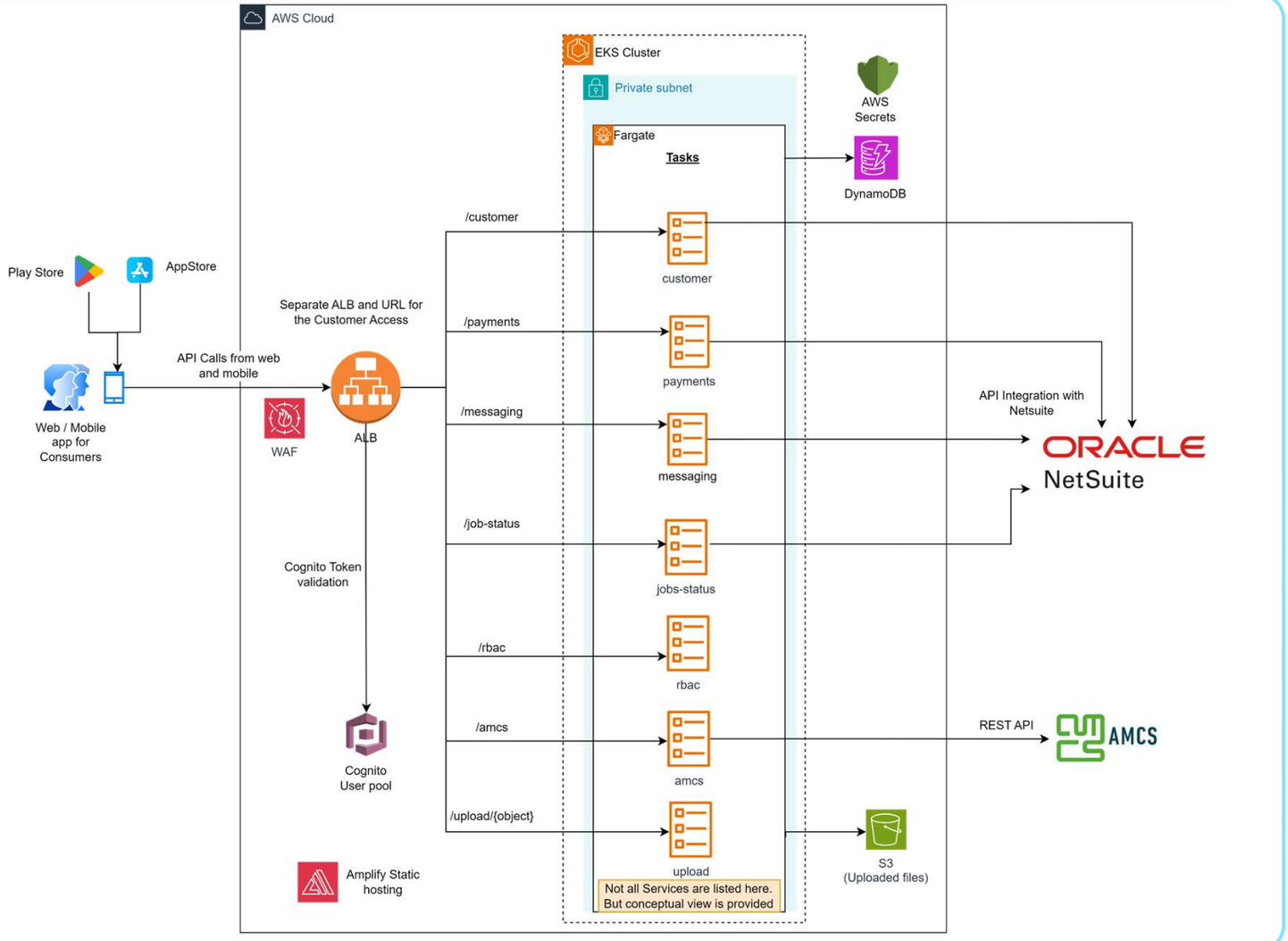
- Additional subdomains will be created for specific purposes within each environment, exemplified by the development environment:
  - **api.dev.prioritywaste.com** will point to the Application Load Balancer (ALB) for API traffic.
  - **app.dev.prioritywaste.com** will point to AWS Amplify for hosting static content.
  - **graph.dev.prioritywaste.com** will point to AWS AppSync for handling GraphQL queries.

TRIANZ℠ | ACCELERATING DIGITAL EVOLUTION

Customer Access

Customer Access (Web and Mobile App)

- Customers can access services through both web and mobile applications.
- The static content for web access will be hosted on AWS Amplify.
- Mobile applications are available on the Google Play Store and Apple App Store.
- The following services will handle all API operations required for customer access via web and mobile applications:
    1. Customer Data Service: Manages customer-related information such as sites, addresses, and contact details.
    2. Payments Service: Handles transactions related to payments, billing, and invoices.
    3. Jobs and Status Service: Provides a list of all jobs, their status, and related metadata.
    4. Messaging Service: Enables chat communication with customers.
    5. RBAC (Role-Based Access Control): Manages access and privileges for users. This service may be required in the future.
- All APIs will be authenticated through Amazon Cognito. However, this is only applicable for active users listed as contacts in the CRM, identifiable via email. There will be no local user database in AWS. The user is checked for record in the database both at the time of registration and login using Lambda hooks in Cognito.
- The majority of the data required for the portal is stored in NetSuite. Consequently, all microservices will need to invoke NetSuite APIs to retrieve data.
- Authentication for NetSuite API calls will be managed via API keys.
- AMCS Integration – To be elaborated later



Microservice details - customer

Purpose:
This microservice is designed to manage customer records within our system. It will handle read and update operations for customer data and ensure data consistency and integrity across the platform.

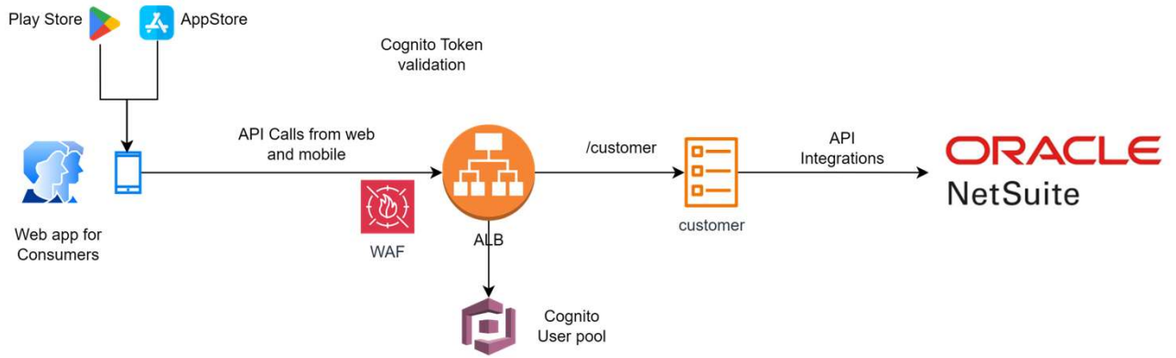
Data model: Data model and the fields will be leveraged from Netsuite platform transparently. There will be no data stored in AWS from this microservice (except logging of the microservice itself). All the critical logging has to be ensured for all microservices to record all transactions.

Integrations: This service will integrate with the Oracle Netsuite Platform for transactions

Code Repository: The code for this will be stored in bitbucket.

Deployment: Deployment of this service will be through the DevOps pipelines in DevOps account.

Infrastructure requirements: There is no persistent storage attached to this microservice. Minimal resources are allotted to this service in Dev and UAT environment. Sizing and autoscaling parameters for production will be decided post development.



API end points:

API path will be synonymous to the Netsuite API path with the suffix mentioned below.
- /customer

Further low level details to be planned during implementation.

Microservice details - Payments

Purpose:
This microservice is designed to provide payment, billing and invoices related transactions to the customers. It will interact with the Netsuite platform, and all the payments are executed and records stored in Netsuite. This microservice will only form a bridge between the front end and the Netsuite platform.

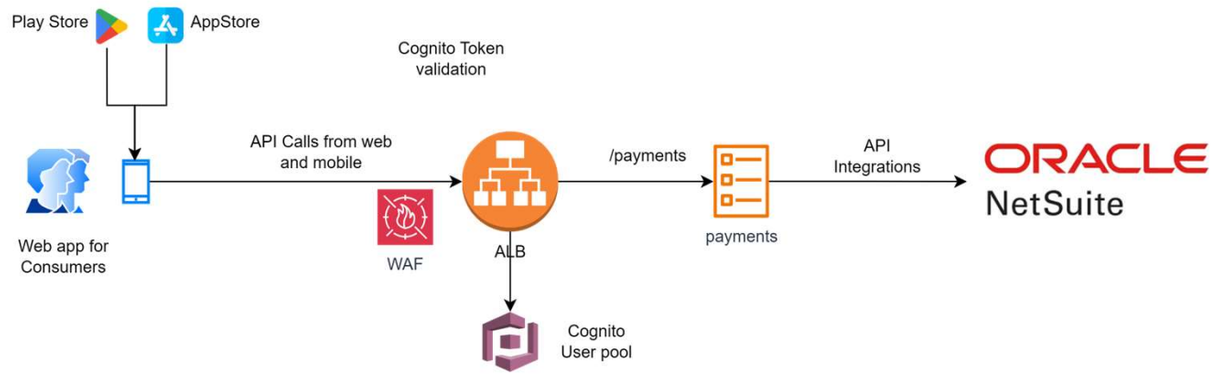
Data model: Data model and the fields will be leveraged from Netsuite platform transparently. There will be no data stored in AWS from this microservice (except logging of the microservice itself). All the critical logging has to be ensured for all microservices to record all transactions.

Integrations: This service will integrate with the Oracle Netsuite Platform for transactions

Code Repository: The code for this will be stored in bitbucket.

Deployment: Deployment of this service will be through the DevOps pipelines in DevOps account.

Infrastructure requirements: There is no persistent storage attached to this microservice. Minimal resources are allotted to this service in Dev and UAT environment. Sizing and autoscaling parameters for production will be decided post development.



API end points:
API path will be synonymous to the Netsuite API path with the suffix mentioned below.
- /payments

Further low level details to be planned during implementation.

Microservice details – jobs_status

Purpose:
The jobs_status microservice is designed to create, update, and provide the status of jobs for customers. It acts as a bridge between the front end and the Netsuite platform, where all job-related records are stored.

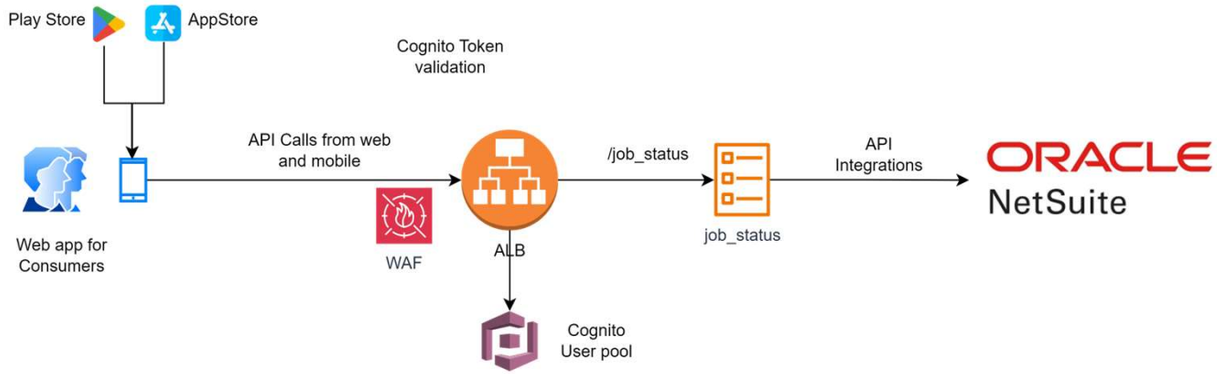
Data model: The data model and fields will be transparently leveraged from the Netsuite platform. No data will be stored in AWS from this microservice, except for logging purposes. Critical logging must be ensured for all microservices to record all transactions.

Integrations: This service will integrate with the Oracle Netsuite Platform for transactions.

Code Repository: The code for this will be stored in bitbucket.

Deployment: Deployment will be managed through DevOps pipelines in the DevOps account.

Infrastructure requirements: There is no persistent storage attached to this microservice. Minimal resources are allocated for the Dev and UAT environments. Sizing and autoscaling parameters for production will be determined post-development.



API end points:
API path will be synonymous to the Netsuite API path with the suffix mentioned below.
- /job_status

Further low level details to be planned during implementation.

Microservice details – messaging

Purpose:
The messaging service is designed to support messaging on the front end and store data in DynamoDB. It will also retrieve information from the Netsuite platform, such as user details and specific job details, to process the messaging..

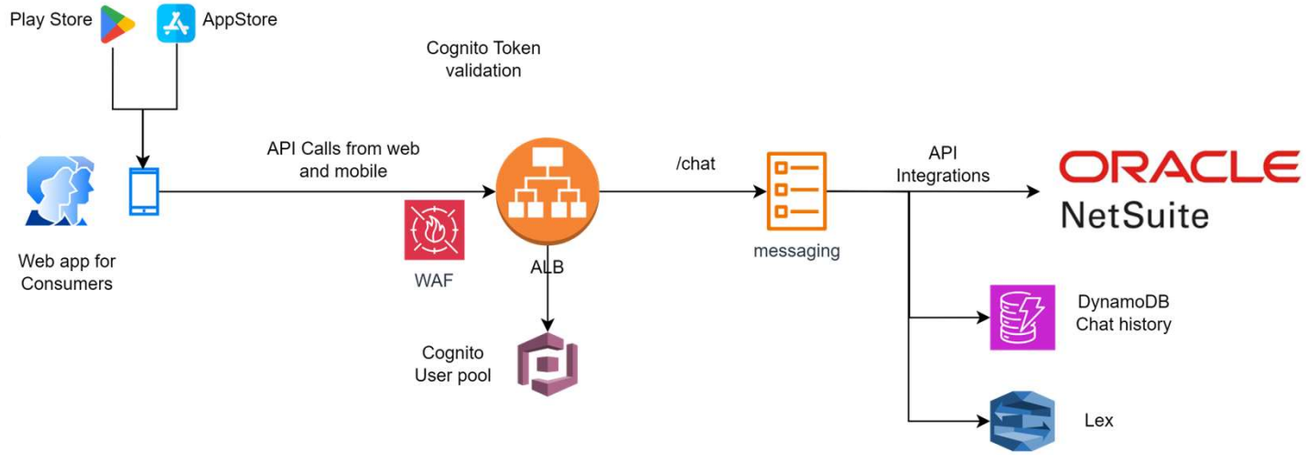
Data model: The data model will leverage fields from both DynamoDB and the Netsuite platform. No data will be stored in AWS from this microservice, except for logging purposes. Critical logging must be ensured for all microservices to record all transactions.

Integrations: This service will integrate with the Oracle Netsuite Platform for transactions. It will connect with DynamoDB for chat history. An integration with Bedrock is planned for future.

Code Repository: The code for this will be stored in bitbucket.

Deployment: Deployment will be managed through DevOps pipelines in the DevOps account.

Infrastructure requirements: There is no persistent storage attached to this microservice. Minimal resources are allocated for the Dev and UAT environments. Sizing and autoscaling parameters for production will be determined post-development.



API end points:
API path will be start with following suffix.
- /chat

e.g. /chat/getmessages:
To list the recent 10 chat messages for the current user.
The response field will be picked from the DynamoDB data schema.

Further low-level details to be planned during implementation.

Microservice details – rbac

Purpose:
The RBAC microservice is designed to manage role-based access control for various services. It ensures that users have the appropriate permissions to access specific resources and perform certain actions.

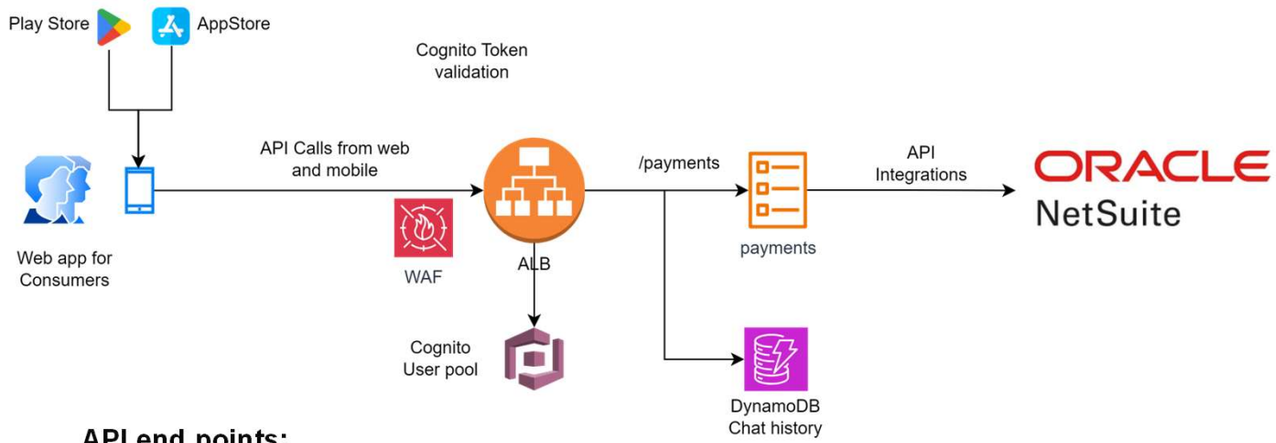
Data model: The data model will leverage fields from both DynamoDB and the Netsuite platform. No data will be stored in AWS from this microservice, except for logging purposes. Critical logging must be ensured for all microservices to record all transactions.

Integrations: This service will integrate with the Oracle Netsuite Platform for transactions. It will connect with DynamoDB for rbac information.

Code Repository: The code for this will be stored in bitbucket.

Deployment: Deployment will be managed through DevOps pipelines in the DevOps account.

Infrastructure requirements: There is no persistent storage attached to this microservice. Minimal resources are allocated for the Dev and UAT environments. Sizing and autoscaling parameters for production will be determined post-development.



API end points:

/rbac:
To list the all the privileges for the current user.
The fields in the DynamoDB will be leveraged for the response.
Further low level details to be planned during implementation.

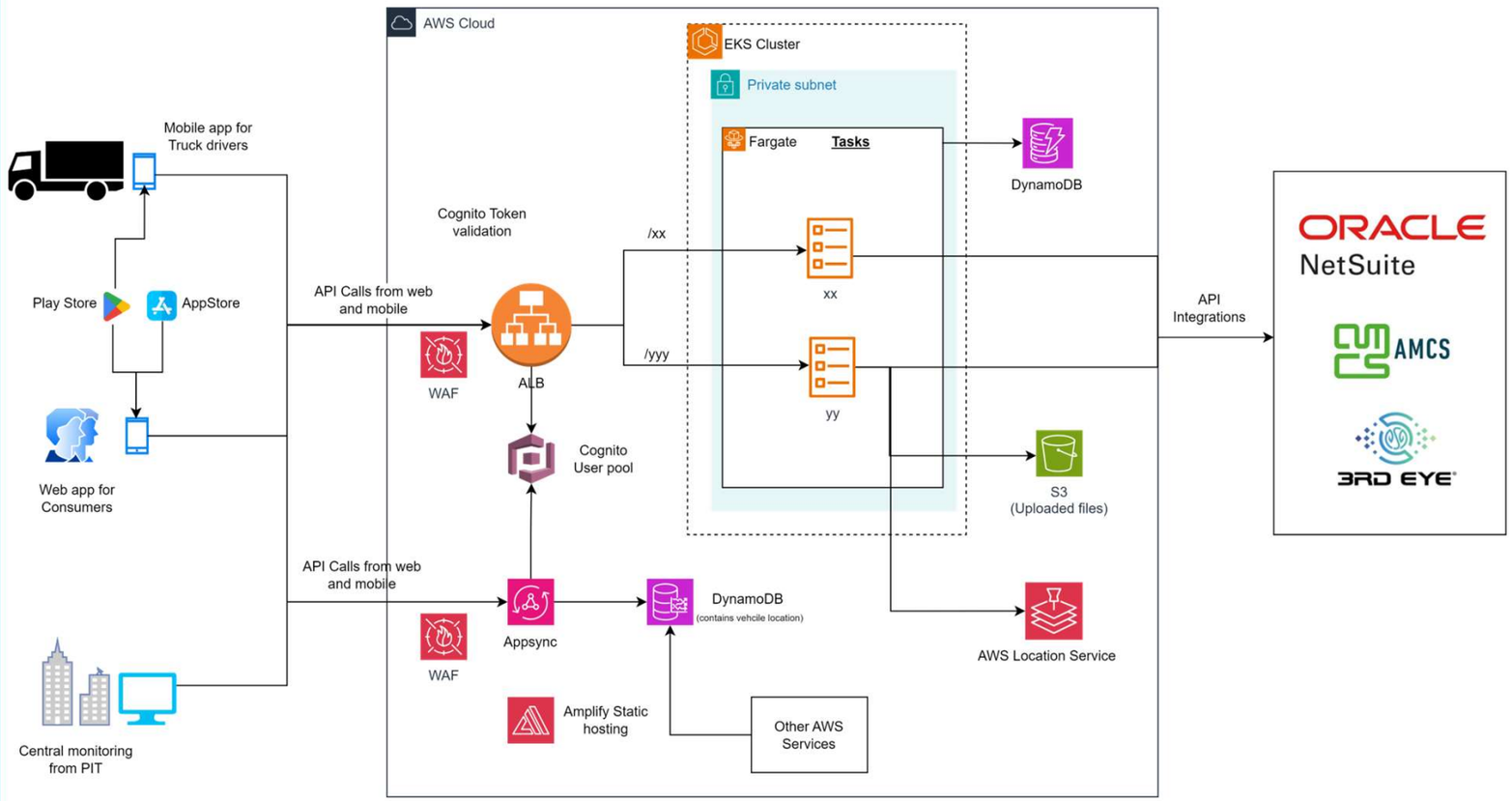
TRIANZ℠ | ACCELERATING DIGITAL EVOLUTION

Security Design

www.trianz.com

AWS WAF

- Critical services exposed to Internet (e.g. ELB) must be protected using AWS WAF.
- Services which must have the WAF enable include
  - All instances of Cloud front distribution, API Gateway, Application Load Balancer, Appsync GraphQL API, Cognito User pool
- Following AWS managed rules must be leveraged (sorted by each services
- Common rules for all services
  - All baseline rule groups, IP Reputation rule, AWS WAF BOT Control rule group
- Cognito Userpool
  - AWS WAF Fraud Control account creation fraud prevention (ACFP) rule group,
  - AWS WAF Fraud Control account takeover prevention (ATP) rule group
  - SQL database managed rules
- Application Load Balancer and API Gateway
  - SQL Database managed rules
- WAF rule should block the access originating from specific countries (list to be provided by Priority)



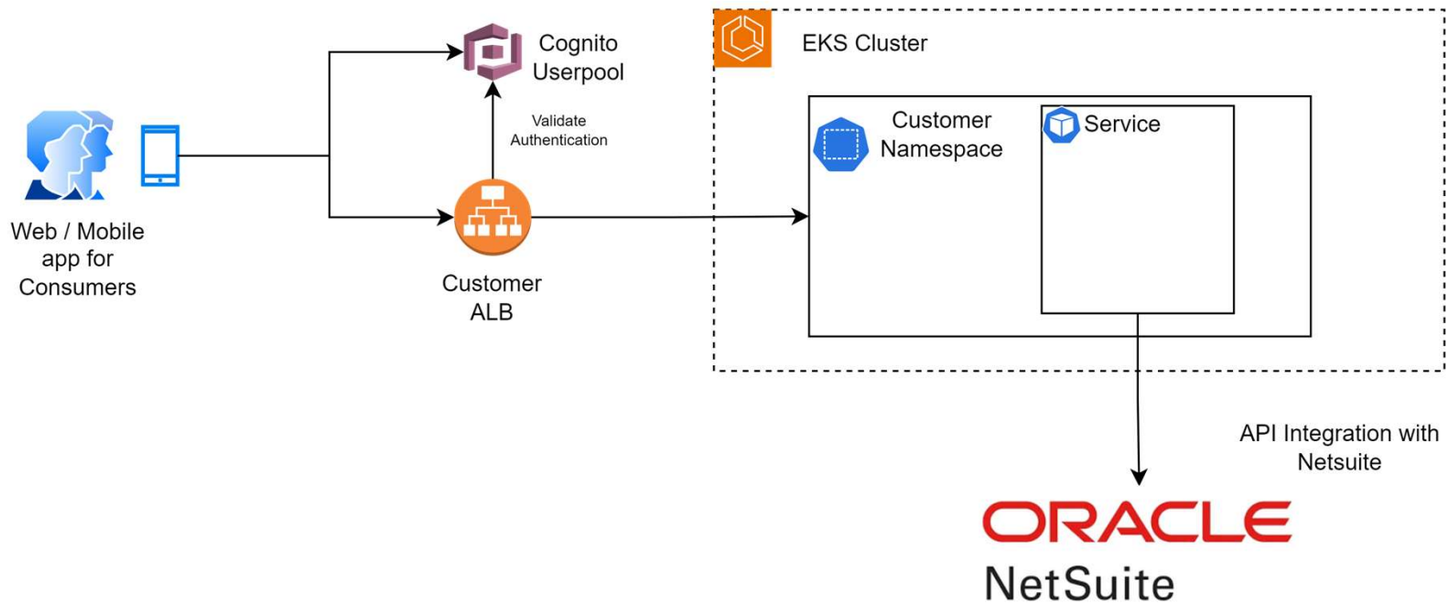
Customer Authentication and authorization flow

Authentication

- The client sends an HTTP request to the ALB endpoint without authentication-session cookies.
- The ALB redirects the request to the Amazon Cognito authentication endpoint.
- The client is authenticated by Amazon Cognito.
- The client is directed back to the ALB with the authentication code.
- The ALB uses the authentication code to obtain the access token from the Amazon Cognito token endpoint.
- The ALB also uses the access token to get the client's user claims from the Amazon Cognito UserInfo endpoint.
- The ALB prepares the authentication session cookie containing encrypted data.
- The ALB redirects the client's request with the session cookie.
- The client uses the session cookie for all further requests.
- The ALB validates the session cookie and decides if the request can be passed through to its targets.
- The validated request is forwarded to the backend services inside the EKS cluster.
- The ALB adds HTTP headers that contain data from the access token and user-claims information.

Authorization

- In case of customer access, the role of the customer is available in Netsuite (billing contact, site contact etc). The user is given only the needed access based on his role in Netsuite.



Customer identity lifecycle management

Registration

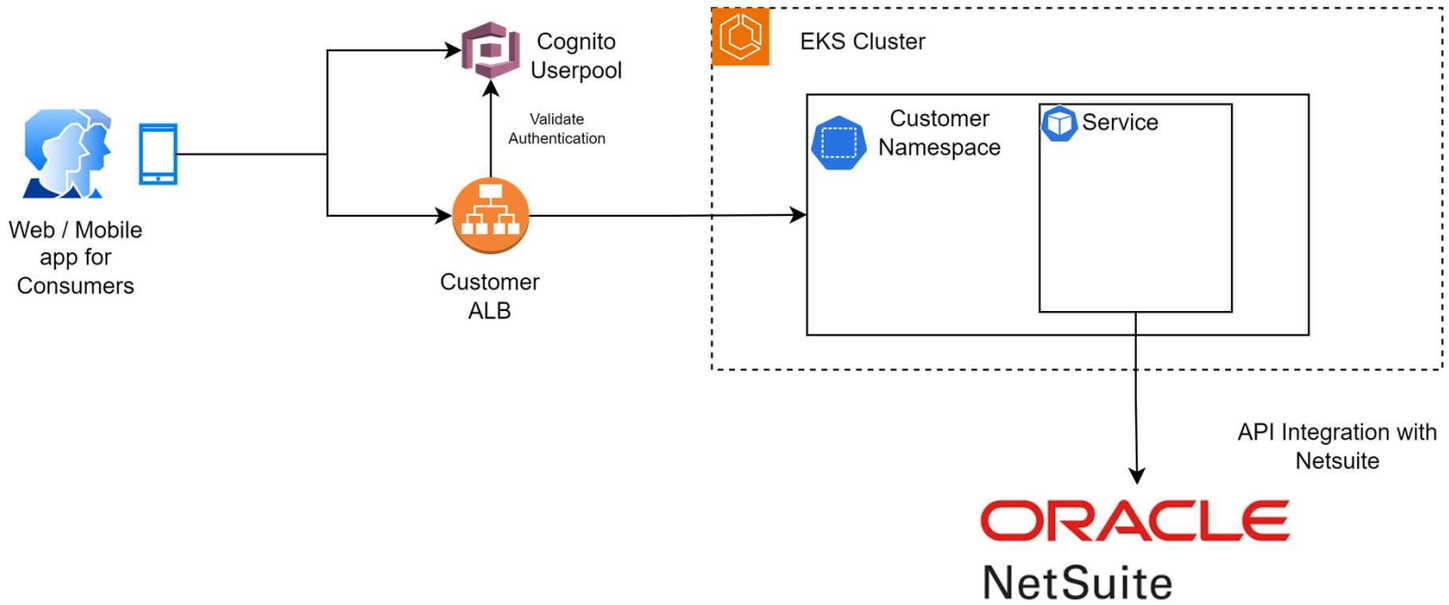
- During the registration process, all users are validated against Netsuite using their invoice number, customer ID, and email address.
- Upon finding a matching record in Netsuite, the user is permitted to complete the registration and log in.

Validation

- Each time a user logs in, their status in Netsuite is verified.
- If the user is active and the company they belong to is also active, the user is granted access to the application.

Clean-up

- On a monthly basis, a scheduled job must verify the active users in Cognito by comparing them against the database in Netsuite.
- If either the company or the user is found to be inactive, the user is deleted from Cognito.
- Following this clean-up, if the user needs to log in again, they must undergo the complete registration process once more.



TRIANZ℠ | ACCELERATING DIGITAL EVOLUTION

Great Place To Work. Certified 2023 USA | INDIA

Thank you

yaseen.mohammed@trianz.com